Gra Stackelberga

Marcin Kocot

Zadanie wykonano w języku Kotlin, przy użyciu build-toola Bazel.

Scenariusze

Utworzono skrypt w języku Python, odpowiedzialny za generację losowych scenariuszy. Generuje on plik JSON ze wszystkimi danymi:

- Nazwy zasobów, ataków i obron (losowe, ale gramatycznie poprawne - przykładowo `"Promise the labored quartz"`)
- Koszty obron i ataków (losowa liczba rzeczywista z zakresu `[0, 100)`)
- Wartość zasobów (z zakresu `[90, 390)`)
- Zyski z ataków (zakres `[0, 1000)`)
- Szanse powodzenia ataku przy danej obronie na danym zasobie

Zakresy zostały wyznaczone empirycznie, aż do uzyskania ciekawych scenariuszy - zbyt wysoka cena lub zbyt niski zysk sprawiał, że atak/obrona traciły sens. Największy przetestowany scenariusz ma po dziesięć zasobów, obron i ataków.

Obliczanie wyników

Rozwiązanie stare

Pierwsze rozwiązanie cechowało się bardzo słabą złożonością (`2 ^ ((A + D) * R)`). Sprawiało to, że nawet dla konfiguracji 3-3-3 (Ataki-Obrony-Zasoby) program potrzebował ponad 250 tysięcy iteracji, a dla konfiguracji 10-10-10 było to ponad półtora decyliona.

Rozwiązanie nowe

Istotnym olśnieniem był fakt, że kombinacje dobranych ataków i obron są niezależne od siebie pomiędzy zasobami - obrona zasobu A nie wpływa na wybrane ataki na zasób B. Dzięki temu odkryciu, złożoność została obniżona do `R * (2 ^ (A + D))`. Dla konfiguracji 10-10-10 z decyliona iteracji robi się dziesięć milionów.

Dla każdego zasobu z osobna program iteruje po wszystkich kombinacjach obron. Dla każdej z nich analizuje wszystkie możliwe kombinacje ataków. Dla każdej pary kombinacji obron i ataków liczone są następujące wartości:

1. Szansa na sukces każdego ataku (iloczyn szans na sukces ataku przeciw każdej aktywnych obron)

2. Szansa na porażkę wszystkich ataków (iloczyn szans na **porażkę** każdego z ataków z osobna)

3. Wypłata strony atakującej ze strategii (iloczyn zysku z ataku i szansy na sukces, minus suma kosztów ataku)

Przykładowo mając dwie obrony i dwa ataki, każdy ze skutecznością 50%, kosztem 10 i zyskiem 50, uzyskamy:

1. Każdy atak ma `50% * 50% = 25%` szans na sukces

2. Dwa ataki wspólnie mają szansę na porażkę `75% * 75% = 56,25%`

3. Atakujący otrzyma wypłatę `25 * 43,75% - (10 + 10) = 21,875 - 20 = 1,875`

Atakujący wybiera najkorzystniejszą strategię, a obrońca po przetestowaniu wszystkich obron podejmuje analogiczną decyzję dla siebie (do obliczenia swojej wypłaty używając wartości zasobu, kosztów obrony i szansy na porażkę ataków).

Wyświetlanie wyników

Przykładowy wynik wykonania programu, dla konfiguracji 10-10-10 (dla oszczędności miejsca w sprawozdaniu podano wyniki tylko dla trzech zasobów):

```
Total Defence Payoff: $1093.53
Total Attack Payoff: $2628.61
Defence Plan:
  Resource 'res1' - The statuesque shade with a wasp
    | Will not be defended
    | Will be attacked by:
    |     Detect the unbecoming descendant ('atk9')
  Resource 'res8' - The weary celebrity
    | Will be defended by:
    |     Stick the spurious focus ('def9')
    |     Count the picayune subtitle ('def7')
    |     Want the fast casualty ('def10')
    | Will be attacked by:
    |     Detect the unbecoming descendant ('atk9')
    |     Man the historical waffle ('atk8')
    |     Publicize the icky circulation ('atk5')
    |     Squeal the astonishing fratricide ('atk1')
  Resource 'res10' - The silent tadpole
    | Will be defended by:
    |     Promise the labored quartz ('def2')
    |     Copy the tranquil buffer ('def1')
    | Will not be attacked
```